Studio Nx

JS 27 — initLogoutButton()

2026-10-08

JS 27 — `initLogoutButton()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js` (새 함수) · `pages/userinfo.js` (import) · `pages/mypage.js` · `pages/myproject.js` (주석)
작업	P1-10 회원 정보 화면 JS — 왼쪽 메뉴 Log out · PRD 01 FR-105
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	26 `initWithdraw()` · 28 `initContactForm()` (P1-12)

1. 하는 일

왼쪽 메뉴의 **Log out** 을 누르면 버튼을 잠그고 로그아웃한 뒤 홈으로 간다.

누른 뒤	결과
바로	버튼 잠김 (두 번 눌러도 요청은 한 번)
서버	`POST /api/v1/auth/logout` → 리프레시 쿠키 삭제 (P1-05 34)
화면	메모리 토큰 삭제 → `home.html` (헤더 Sign In · Join Us), 뒤로 가기로 돌아오지 않음

실제 일은 이미 있는 P1-02 08 `logout()` 이 한다 — 이 함수는 버튼에 연결하고 잠그는 일만.

2. 언제 쓰나

화면	버튼	지금
userInfo.html	`#logout-button`	✅ 이번에 연결 (`userinfo.js` 맨 아래 `initLogoutButton();`)
myPage.html	같은 `#logout-button`	P1-17 때 import 만 하면 됨 (`mypage.js` 템플릿 주석에 적음)
myProject.html	같은 `#logout-button`	P1-14 때 import 만 하면 됨 (`myproject.js` 주석)

세 화면 공통이라 `core/auth-ui.js` 에 하나만 두었다. 템플릿은 페이지마다 같은 함수를 하나씩 두었다 (`userinfo.js` · `mypage.js` · `myproject.js`). `userinfo.js` 의 빈 함수는 지우고 import 로 바꿨다. 나머지 두 페이지의 빈 함수는 그 작업(P1-14 · P1-17) 때 지운다.

3. 코드

import (`auth-ui.js` 맨 위)

```
import { api, setAccessToken } from './api.js';
import { isLoggedIn, parseLoginHash, logout } from './auth.js';
```

`logout` 을 이번에 추가했다. `userinfo.js` 는 `import { …, initLogoutButton } from '../core/auth-ui.js';` 로 바꾸고, 쓰지 않게 된 `logout` import 를 뺐다.

함수

```
export function initLogoutButton(button = document.getElementById('logout-button')) {
  if (!button || button.dataset.logout === 'on') return;
  button.dataset.logout = 'on';
  button.addEventListener('click', () => {
    button.disabled = true;
    logout(); // 끝나면 다른 페이지로 옮겨 가므로 다시 켤 필요 없음
  });
}
```

최신 문법으로 쓰면 (공부용)

```
export function initLogoutButton(button = document.getElementById('logout-button')) {
  if (button?.dataset.logout === 'on' || !button) return;
  button.dataset.logout = 'on';
  button.addEventListener('click', () => { button.disabled = true; logout(); }, { once: true });
}
```

최신 문법	실제 코드	뜻
`button?.dataset…`	`!button || …` (먼저 없는지 봄)	버튼이 있을 때만
`{ once: true }`	`button.disabled = true` 로 막음	한 번만 받기 — 잠금 표시가 화면에도 보이는 쪽을 택함

문법 풀이

식	뜻
`(button = document.getElementById('logout-button'))`	**매개변수 기본값** — 넘기지 않으면 `#logout-button`. 다른 버튼에도 쓸 수 있게
`button.dataset.logout === 'on'`	이미 연결했는지 (P1-07 10 비밀번호 토글과 같은 방법)
`logout();` (`await` 없음)	기다릴 필요가 없다 — 끝나면 페이지가 바뀐다

4. 흐름

1. 버튼이 없거나 이미 연결했으면 끝.
2. 누르면 → 버튼 잠금 → `logout()` (서버 쿠키 삭제 → 메모리 토큰 삭제 → 홈).

5. 왜 이렇게

선택	이유
공용 파일에 하나만	세 화면의 버튼 · 동작이 똑같다. 템플릿처럼 세 번 쓰면 고칠 곳도 셋
`auth-ui.js` 에	로그인 · 헤더 상태 등 "인증 화면 동작" 을 모아 둔 곳 (P1-07)
잠금	느린 망에서 여러 번 눌러 로그아웃 요청이 여러 번 가지 않게
두 번 연결 방지	페이지가 실수로 두 번 불러도 요청은 한 번 (시험으로 확인)
확인창 없음	로그아웃은 되돌릴 수 있다 (다시 로그인). 확인창은 되돌릴 수 없는 일(탈퇴 · 삭제)에만

6. 시험 (실제 userInfo.html · 실제 서버 · 5개)

시험	확인한 것
연결	`data-logout="on"`, 한 번 더 불러도 연결 안 늘어남
누름	바로 버튼 잠김 → 로그아웃 요청 **1번**
옮겨 간 뒤	`/home.html`, 방문 기록 수 그대로, `isLoggedIn()` false, 헤더 로그아웃 상태

5/5 통과. `auth-ui.js` 를 고쳤으므로 그 파일을 쓰는 P1-07 시험(09 · 10 · 12 · 13)과 P1-10 시험(22 ~ 25)도 다시 돌려 모두 통과 (시험이 만든 계정은 지움).

처음 한 번은 **시험 쪽 문제**로 결과를 받지 못했다 — 로그아웃이 너무 빨라 시험 코드가 끝나기 전에 페이지가 바뀌었다. 요청 수를 `sessionStorage` 에 바로 적고, 누른 직후 끝내도록 고쳤다.